Python-Pydantic-M-H-Set2

FDE / Architect practice | Reasoning, alternatives, and design takeaways

01. An await splits a read-modify-write operation

Answer C: 1; protect the whole read/await/write region with one shared asyncio.Lock.

Each task reads 0 before yielding at sleep(0). Both later write 1, so one increment is lost. A single shared lock around the complete operation serializes access. Where possible, redesign the operation to avoid holding a lock across slow I/O; for state shared across processes, an asyncio lock is insufficient.

Why the alternatives fail

A. Different lock instances do not exclude each other.

B. A task can be interleaved with another task at an await even on one thread.

D. gather runs these coroutines concurrently; argument order does not serialize their writes.

Design takeaway: Reason about atomic operations, not just thread count.

Official references: [1]

02. Related tasks need an owned failure scope

Answer A: Create the three tasks inside an async with asyncio.TaskGroup() block.

TaskGroup provides a lifetime and failure scope for related tasks. A non-cancellation failure cancels unfinished siblings, waits for their cleanup, and then raises an exception group. The caller must handle that grouped failure appropriately. This is distinct from merely launching tasks concurrently.

Why the alternatives fail

B. Default gather propagates the first exception without automatically cancelling the other awaitables.

C. Observing one failure does not cancel and join the other tasks.

D. return_exceptions=True collects failures as results while allowing siblings to keep running.

Design takeaway: Concurrency needs explicit ownership of completion and failure.

Official references: [2]

Python-Pydantic-M-H-Set2

FDE / Architect practice | Reasoning, alternatives, and design takeaways

03. Cancellation is not a successful fallback

Answer D: After release(), use bare raise in the CancelledError handler.

Returning from the handler suppresses the CancelledError and makes the coroutine finish with a value. A bare raise preserves cancellation after cleanup. Converting it into an unrelated failure changes the meaning for callers, while shielding changes which work is cancelled rather than honoring the requested contract.

Why the alternatives fail

A. Explicitly catching and returning from the handler can suppress cancellation.

B. RuntimeError signals an ordinary failure, not the cancellation state the caller expects.

C. Shielding protects inner work from caller cancellation and does not repair this suppressed cancellation.

Design takeaway: Release resources on cancellation, then preserve the control signal.

Official references: [2]

04. A request budget spans all stages

Answer B: Wrap both awaits in one asyncio.timeout(2) block and catch TimeoutError outside it.

A single timeout scope covers the combined operation. After retrieval consumes about 1.4 seconds, only about 0.6 seconds remain for generation. Timeouts use cancellation, so cleanup and scheduling can add overhead; this is not a guarantee of exact wall-clock termination for arbitrary blocking code.

Why the alternatives fail

A. Independent 2-second limits allow both stages to succeed after roughly 2.6 seconds in this case.

C. A retrospective measurement reports the budget violation after the work has already run.

D. A fresh generation budget ignores time already spent and permits the same total overrun.

Design takeaway: Propagate one deadline through dependent work rather than resetting it at every boundary.

Official references: [2]

05. Backpressure must reach the producer

Answer B: A bounded asyncio.Queue, a fixed worker pool, and producers that await each queue.put().

When the bounded queue fills, an awaited put suspends the producer, slowing ingestion to the consumers' capacity. Fixed worker count also bounds active processing. The bound applies to this pipeline under the stated fixed-size record assumption; external buffering needs its own policy.

Why the alternatives fail

A. Unbounded pending put tasks retain their records outside the queue, defeating the memory bound.

C. The semaphore limits active workers but leaves an unlimited queue of waiting records.

D. Each waiting task can retain a record even before it acquires the semaphore.

Design takeaway: Bound waiting work as well as active work.

Official references: [3]

06. Cancelling an await does not stop a running thread

Answer D: The thread may finish the write; reconcile the operation outcome before assuming it did not happen.

Cancelling the await does not forcibly terminate an already running Python thread. The external write may still succeed even though the request stops waiting. The application needs a way to discover the eventual outcome, such as an operation ID and status lookup, before deciding what further action is safe.

Why the alternatives fail

A. The awaiting task receives cancellation; to_thread does not fabricate a successful business result.

B. Python futures cannot cancel execution that is already running in a thread.

C. Local task cancellation is not a transaction rollback protocol for the remote service.

Design takeaway: Distinguish stopping observation from stopping execution.

Official references: [4], [2]

07. A child task receives a context snapshot

Answer A: A, because create_task copies the current context at task creation.

The child task receives a copy of the current context when create_task is called. That context contains A. The parent's subsequent set to B changes the parent context, not the child's stored binding. This isolates bindings; it does not deep-copy a mutable object stored as a context value.

Why the alternatives fail

B. ContextVar bindings are context-local rather than live references to the parent's later assignments.

C. The default task-creation behavior copies the current context.

D. get returns the current value in one context, not the history of assignments.

> **Design takeaway:** Set request context before launching child work and keep context values safe to share.

Official references: [5], [2]

08. A local semaphore is not a fleet-wide limit

Answer C: Up to 32; use coordinated shared admission control or fixed per-process budgets totaling 8.

Each process owns eight independent permits, so four saturated processes can allow 4 x 8 = 32 calls. A global cap requires coordination or a static allocation whose total is eight. Static budgets sacrifice flexibility and need revision when worker count changes.

Why the alternatives fail

A. An event loop can keep many I/O operations in flight while they are waiting.

B. An asyncio.Semaphore is an in-process synchronization primitive, not distributed shared state.

D. Matching Python variable names do not make objects shared across process address spaces.

> **Design takeaway:** State the scope of every limit: task, process, host, or fleet.

Official references: [1]

Python-Pydantic-M-H-Set2

FDE / Architect practice | Reasoning, alternatives, and design takeaways

09. CPU parallelism depends on the runtime

Answer A: Submit the transformations to a suitably sized ProcessPoolExecutor.

Separate processes have separate interpreters and can execute Python bytecode on multiple cores. The stated workload makes process-transfer overhead a reasonable tradeoff. This conclusion is scoped to GIL-enabled CPython and pure-Python work; native libraries that release the GIL and other runtimes can behave differently.

Why the alternatives fail

B. Declaring a function async does not make its CPU work parallel or automatically nonblocking.

C. Threads generally do not provide parallel pure-Python bytecode execution under this GIL assumption.

D. Yielding improves scheduling opportunities but does not create multicore parallelism.

Design takeaway: Choose an execution model from measured workload characteristics.

Official references: [4]

10. Early exit must close an async stream deterministically

Answer C: Use async with contextlib.aclosing(stream()) as rows, then async for inside that block.

aclosing awaits the generator's aclose() when its context exits, including early exit from iteration. This runs the generator's finally block in the intended task context. It makes resource release part of the caller's explicit control flow rather than relying on eventual finalization.

Why the alternatives fail

A. Async generators use an awaitable aclose(), not a synchronous close() method.

B. Breaking iteration does not itself guarantee immediate async-generator closure.

D. Eventual finalization does not meet the requirement to release the connection before scope exit.

Design takeaway: Make partial consumption an explicit resource-lifecycle case.

Official references: [6]

Python-Pydantic-M-H-Set2

FDE / Architect practice | Reasoning, alternatives, and design takeaways

11. An event tag selects the only valid branch

Answer D: A discriminated union of the two models, with Literal tags and discriminator="status".

The discriminator selects the success branch because the payload says ok. That branch requires value and forbids message, so this inconsistent payload fails. A single model with independent optional fields permits combinations that violate the intended event states unless additional invariants are enforced.

Why the alternatives fail

A. An unconstrained dictionary and prompt do not enforce the event-state contract.

B. Independent optional fields allow the inconsistent combination unless a separate invariant is added.

C. Rewriting the status conceals contradictory producer data instead of rejecting it.

Design takeaway: Encode valid states directly so contradictory combinations fail at the boundary.

Official references: [7]

12. Copy-with-update is not revalidation

Answer B: bad str; merge into plain data and call Config.model_validate on the merged data.

model_copy applies update values without validating them. The copied retries therefore becomes the string bad despite the annotation and constraint. For an external patch, validate the merged representation before using it, for example Config.model_validate({**old.model_dump(), **patch}). Deep copying only changes object-copy behavior.

Why the alternatives fail

A. model_copy explicitly does not validate its update data.

C. No parser runs here, and Pydantic does not silently substitute zero for this malformed integer.

D. deep=True copies nested objects; it does not add validation to update values.

Design takeaway: Treat trusted transformation helpers and untrusted-input validators as different boundaries.

Official references: [8]

Python-Pydantic-M-H-Set2

FDE / Architect practice | Reasoning, alternatives, and design takeaways

13. Strict JSON and strict Python inputs differ

Answer D: The Python-dict call fails and the JSON call succeeds; test the actual input mode used in production.

Strict Python validation expects a date instance here. Strict JSON validation can accept a date string because JSON has no native date type. Strictness therefore does not mean identical acceptance across Python and JSON inputs. An API test that validates a pre-parsed dictionary can exercise a different boundary from model_validate_json.

Why the alternatives fail

A. JSON can carry the supported date string representation even though it lacks a native date type.

B. Strict JSON mode permits representations needed for types that JSON cannot encode natively.

C. Strict Python-mode date validation rejects the string instead of coercing it.

Design takeaway: Validate the transport-specific contract, not an assumed equivalent representation.

Official references: [9]

14. A frozen model can contain mutable state

Answer A: Append succeeds; use tuple[str, ...] for tags as well as the frozen model.

frozen prevents assigning a new value to a model attribute; it does not recursively freeze the existing list object. The list can still mutate through append. A tuple of strings removes this nested mutation path, giving this simple snapshot an immutable representation of its tags.

Why the alternatives fail

B. The freeze applies to model attribute assignment, not arbitrary operations on nested objects.

C. Extra-field policy affects validation of unknown input keys, not list mutability.

D. append mutates the referenced list in place and does not construct another model.

Design takeaway: Immutability is a property of the whole object graph.

Official references: [10]

15. A previously valid instance can become invalid

Answer B: The invalid instance is accepted; set revalidate_instances="always" on Config.

Default assignment behavior permits this mutation, and the default revalidation policy trusts existing instances of the model. Setting revalidate_instances="always" makes the second boundary validate the instance and reject the bad field. validate_assignment=True is a separate option that would catch this direct assignment earlier.

Why the alternatives fail

A. An existing model instance is not revalidated by default.

C. Direct assignments are not validated unless that behavior is explicitly enabled.

D. Extra-field rejection does not revalidate values on an already-created instance.

Design takeaway: A type-correct object at creation is not proof of validity after mutation.

Official references: [11]

16. Serialization follows the declared model boundary

Answer C: Only user.name; fields defined only on the subclass are omitted.

Default serialization for a model-typed field uses the schema of the declared type. User declares name but not token, so the token is omitted here. Duck-typed serialization such as serialize_as_any=True can change this behavior. An explicit public response model remains a useful reviewable boundary.

Why the alternatives fail

A. A compatible subclass instance is accepted for this field.

B. Serializing all runtime-subclass fields requires behavior different from the stated default.

D. A plain str named token has no automatic masking policy.

Design takeaway: Review serialization rules before enabling broader runtime-field exposure.

Official references: [12]

17. Validate a collection without a wrapper model

Answer C: Create TypeAdapter(list[Record]) once and call adapter.validate_json(raw) per request.

TypeAdapter validates type expressions such as a list of models directly. Reusing it avoids repeatedly building the same schema and validator. validate_json handles the raw JSON input and returns the validated model collection rather than merely asserting a type to static tooling.

Why the alternatives fail

A. A return annotation does not change json.loads behavior or enforce record fields.

B. cast does not validate data or construct Record instances at runtime.

D. This checks a much looser dictionary shape and does not produce the required Record objects.

Design takeaway: Validate the actual composite type and reuse stable validation machinery.

Official references: [13]

18. Per-request validation rules need explicit context

Answer B: Extraction.model_validate(payload, context={"labels": allowed_labels})

model_validate accepts a context argument that is available through ValidationInfo inside validators. This keeps the validation policy attached to the specific call. The validator must still explicitly check membership and define what happens when context is missing; passing context alone enforces nothing.

Why the alternatives fail

A. The ordinary model constructor does not pass this keyword as validator context; it is input data.

C. Adding a context key to the payload is not the same as supplying validation context.

D. A mutable class-level rule can race across concurrent requests and apply another request's taxonomy.

Design takeaway: Make request-specific policy an explicit input to validation.

Official references: [14]

19. Decimal cannot recover precision already lost upstream

Answer D: False True; retain decimal text and construct Decimal before binary-float arithmetic.

The float addition has already produced a binary approximation before Decimal receives it. Decimal faithfully converts that float value; it cannot infer that the intended result was exactly 0.3. Constructing Decimal from the original strings preserves decimal values, so the second sum compares equal to Decimal("0.3").

Why the alternatives fail

A. The string-based Decimal computation is the exact one in this example.

B. Conversion preserves the supplied float value, including its approximation, rather than reconstructing lost intent.

C. Decimal represents these finite base-10 fractions exactly under the stated context.

Design takeaway: Choose numeric representation before performing arithmetic, not just at final serialization.

Official references: [15]

20. Schema evolution needs a deployable compatibility sequence

Answer A: First deploy consumers that validate both explicit versions and normalize them; confirm all readers are upgraded, then enable V2 producers.

Compatible readers must exist everywhere that new events can arrive before writers emit the new format. Version-specific schemas can normalize to one internal representation, with explicit unit conversion and an agreed currency rule for legacy events. Keep V1 reading support until old queued and retained events no longer need it.

Why the alternatives fail

B. Old consumers can receive V2 during the rollout and reject it before their upgrades arrive.

C. Renaming and type coercion cannot safely infer changed units or currency semantics.

D. Ignoring unknown fields does not supply missing required fields or teach old code new meaning.

Design takeaway: Treat schema changes as a producer-consumer deployment protocol.

Official references: [7], [10]

Python-Pydantic-M-H-Set2

Version assumptions and official sources

Code behavior checked on Python 3.12.14 and Pydantic 2.14.1. Python questions assume ordinary GIL-enabled CPython where relevant. Documentation checked October 28, 2026. Standard imports are omitted from snippets.

The questions and scenarios are original. API behavior is supported by execution checks and the official references below; architecture answers apply those behaviors to the stated constraints. Review the explanation, then articulate why each alternative fails.

[1] Python: asyncio locks and semaphores
https://docs.python.org/3.12/library/asyncio-sync.html

[2] Python: TaskGroup, timeouts, and task contexts
https://docs.python.org/3.12/library/asyncio-task.html

[3] Python: bounded asyncio queues
https://docs.python.org/3.12/library/asyncio-queue.html

[4] Python: futures, cancellation, and process pools
https://docs.python.org/3.12/library/concurrent.futures.html

[5] Python: context variables
https://docs.python.org/3.12/library/contextvars.html

[6] Python: deterministic async-generator cleanup
https://docs.python.org/3.12/library/contextlib.html#contextlib.aclosing

[7] Pydantic: discriminated unions
https://docs.pydantic.dev/latest/concepts/unions/

[8] Pydantic: BaseModel API, copy and construction
https://docs.pydantic.dev/latest/api/base_model/

[9] Pydantic: JSON validation
https://docs.pydantic.dev/latest/concepts/json/

[10] Pydantic: model behavior and immutability
https://docs.pydantic.dev/latest/concepts/models/

Python-Pydantic-M-H-Set2

Version assumptions and official sources

Code behavior checked on Python 3.12.14 and Pydantic 2.14.1. Python questions assume ordinary GIL-enabled CPython where relevant. Documentation checked October 28, 2026. Standard imports are omitted from snippets.

The questions and scenarios are original. API behavior is supported by execution checks and the official references below; architecture answers apply those behaviors to the stated constraints. Review the explanation, then articulate why each alternative fails.

[11] Pydantic: configuration API
https://docs.pydantic.dev/latest/api/config/

[12] Pydantic: serialization modes and field selection
https://docs.pydantic.dev/latest/concepts/serialization/

[13] Pydantic: TypeAdapter
https://docs.pydantic.dev/latest/concepts/type_adapter/

[14] Pydantic: field and model validators
https://docs.pydantic.dev/latest/concepts/validators/

[15] Python: decimal arithmetic
https://docs.python.org/3.12/library/decimal.html